

ServiceNow Client Scripts

Client Scripts Overview

Client Scripts are allowing to user to run Java Script on client side (**Browser Window**) when client based events are occurred or executed

Such as when a form was, **Loads and or submitted form** and when a **field value is changed** and when you **edit cell value from list view**

We can be able to use client scripts to configure **Forms, Form Fields and Field Values** while user is using any particular form.

Client Scripts can do some below activities like

1. Make it as field **Mandatory or Optional**
2. It is used to display message based on **Field Values**
3. Make is as field **Read only and Writable**
4. Make it as filed **Visible and Invisible**
5. Add and modify the options in a choice list based on **user's role**
6. Giving **Info** and **Error Messages** to customer
7. **Add** and **Remove** the choice values
8. **Set** and **Get** values **from fields**
9. Display **Flash** messages
10. Add **Information Message** to customers

Types of Client Scripts

1. onLoad Client Script

An **onLoad Client Script** executes automatically when a form is loaded. It is used to initialize the form and control its behavior before the user interacts with it.

Example Use Cases:

- Display a welcome message.
- Hide fields when the form opens.
- Set default values.

2. onChange Client Script

An **onChange Client Script** executes whenever the value of a specified field changes. It is used to perform actions based on the new value entered by the user.

Example Use Cases:

- Make a field mandatory based on another field's value.
 - Show or hide fields dynamically.
 - Validate entered data.
-

3. onSubmit Client Script

An **onSubmit Client Script** executes when a user submits, saves, or updates a form. It is used to validate data before the record is sent to the server.

Example Use Cases:

- Prevent form submission if required information is missing.
 - Check business conditions before saving.
-

4. onCellEdit Client Script

An **onCellEdit Client Script** executes when a field is edited directly in a list view. It is used to validate or control changes made through list editing.

Example Use Cases:

- Validate list updates.
- Restrict invalid changes in list view.

Here are the commonly used g_form methods in client scripts

- **getValue()** - Gets the value of a field.
- **setValue()** - Sets the value of a field.
- **clearValue()** - Clears the value of a field.
- **getDisplayValue()** - Gets the display value of a field.
- **setMandatory()** - Makes a field mandatory or optional.
- **isMandatory()** - Checks whether a field is mandatory.
- **setReadOnly()** - Makes a field read-only or editable.
- **isReadOnly()** - Checks whether a field is read-only.
- **setDisplay()** - Shows or hides a field.
- **setVisible()** - Makes a field visible or invisible.
- **getLabelOf()** - Gets the label of a field.
- **setLabelOf()** - Changes the label of a field.
- **showFieldMsg()** - Displays a message below a field.
- **hideFieldMsg()** - Hides a field message.
- **hideAllFieldMsgs()** - Hides all field messages.
- **addInfoMessage()** - Displays an information message at the top of the form.
- **addErrorMessage()** - Displays an error message at the top of the form.
- **clearMessages()** - Removes all form messages.

- **flash()** - Highlights a field temporarily.
- **getUniqueValue()** - Gets the Sys ID of the current record.
- **getTableName()** - Gets the name of the current table.
- **getReference()** - Retrieves information from a reference field.
- **isNewRecord()** - Checks whether the record is new.
- **save()** - Saves the current record.
- **submit()** - Submits the form programmatically.
- **setSectionDisplay()** - Shows or hides a form section.
- **getSections()** - Gets all sections on the form.
- **activateTab()** - Opens a specific tab on the form.

Client Scripts Exercises

Exercise: 1

Requirement

When Incident Form Loaded then some fields are make **Mandatory** or **Read Only** or **Visible /Invisible** and add an **Information Message** to define fill all mandatory fields

1. **Number** and **Priority** Fields are **Read only**
2. **Caller** and **Short Description** fields are **Mandatory**
3. **Configuration Item** field is should not be seen (**Invisible**)
4. Display an **Information Message** top on the form if the record is new

Client Script Type

OnLoad

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**
4. Fill Clients Scripts **Form**
5. **Name:** Initial Form Validation
6. **Table:** Incident
7. **UI Type:** All
8. **Type:** OnLoad

Script

```
function onLoad() {  
    if(g_form.isNewRecord()){  
        g_form.addInfoMessage('Please Fill All  
Mandatory Fields');  
    }  
    g_form.setMandatory('caller_id',true);  
  
    g_form.setMandatory('short_description',true);  
    g_form.setDisabled('number',true);  
    g_form.setDisabled('priority',true);  
    g_form.setDisplay('hold_reason',false);  
}
```

9. Open **New Incident** form check the validation

10. **Incident** → Click on **Create New**

Exercise: 2

Requirement

1. When Incident State is **On-Hold** Then On Hold Reason field should be **Visible** and **Mandatory**
2. Add warning message below hold reason field that is “Please select at least one hold reason”

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition**→**Client Scripts**
2. Open **Client Scripts**

3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if(newValue == 3){
        g_form.setMandatory('hold_reason',true);
        g_form.setDisplay('hold_reason',true);

g_form.showFieldMsg('hold_reason','Please select
atleast one hold reason','warning',false);
    }else{

g_form.setMandatory('hold_reason',false);
        g_form.setDisplay('hold_reason',false);
    }
}
```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Create New**
3. Change state value to **On-Hold**
4. Check on hold reason field is **Visible** and **Mandatory**
5. Follow below screen shot for reference

Exercise: 3

Requirement:

Any customer after submitted (Priority 1-Critical) incident

1. We need to display one alert message to customer about **Criticality** (Are you submitting Priority one ticket)
2. Include **Ok** and **Cancel** Buttons also

Client Script Type

OnSubmit

Procedure

1. Navigate to **System Definition**→**Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onSubmit() {  
    var critical = g_form.getValue('priority');  
    if(critical == 1){  
        return confirm(getMessage('Are you  
submitting priority one ticket.'));  
    }  
}
```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Create New**
3. Select **Impact-1** and **Urgency-1**
4. Click on **Submit** Button
5. Follow below screen shot for reference

Exercise: 4

Requirement

1. When Incident state is **Resolved** Then **Resolution Code** and **Resolution Notes** are mandatory
2. **Assigned to** and **Assignment Group** should not be seen

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition**→**Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if(newValue == 6){
        g_form.setMandatory('close_code',true);
        g_form.setMandatory('close_notes',true);
        g_form.setDisplay('assigned_to',false);

g_form.setDisplay('assignment_group',false);
    }else {
        g_form.setMandatory('close_code',false);

g_form.setMandatory('close_notes',false);
        g_form.setDisplay('assigned_to',true);
    }
}
```

```
g_form.setDisplay('assignment_group',true);  
    }  
}
```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Create New**
3. Change state value to **Resolved**
4. Check validation on below screen shot for reference

Exercise: 5

Requirement

1. If Selected **Caller** is **VIP** in Incident table, then **Impact** and **Urgency** field values setup Impact **High** and Urgency **High** then **Priority 1-Critical** should be set automatically
2. **Impact** and **Urgency** fields should be read only on above condition
3. Display on alert message on browser that is **(Caller is VIP!**

Developer TIP: Make one **Caller VIP (Fred Luddy)** from user record (Already we know how to do this)

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    var caller =
g_form.getReference('caller_id');
    if(caller.vip == 'true'){
        alert('Caller is VIP');
        g_form.setValue('impact', 1);
        g_form.setValue('urgency', 1);
        g_form.setDisabled('impact',true);
        g_form.setDisabled('urgency',true);
    }else{
        if(caller.vip == 'false'){
            g_form.setValue('impact', 3);
            g_form.setValue('urgency', 3);
            g_form.setDisabled('impact',false);
            g_form.setDisabled('urgency',false);
        }
    }
}
```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Create New**
3. Select any **VIP Caller** like **Fred Luddy**
4. Check validation on below screen shot for reference

Exercise: 6

Requirement

1. When customer selected Incident **Category** is **Inquiry/Help**, then **Subcategory** field should be hide
2. Remove choice **1-High** from **Impact** and **Urgency** fields
3. Display an **Error Message** below impact field (**Low impact is not preferring high priority**)

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if(newValue == 'inquiry'){
        g_form.removeOption('impact',1);
        g_form.removeOption('urgency',1);
        g_form.setDisplay('subcategory',false);
        g_form.showFieldMsg('impact','Low impact
is not prefer high priority','error',false);
    } else {
        g_form.addOption('impact',1);
        g_form.addOption('urgency',1);
    }
}
```

```
        g_form.setDisplay('subcategory',true);  
    }  
}
```

Exercise: 7

Requirement

1. When current incident state is **New** then remove all choices from **State** fieldChoices need to be removed (**In-Progress, On-Hold, Resolved, Closed, Cancelled**).
2. When current incident **State** is **In-Progress** or **On-Hold** then remove all choices from **State** Field Choices need to be removed (**New, Closed,**).
3. When current incident State is **Resolved**, then remove these all choices Choices need to be removed (**New, In-Progress, On-Hold, Cancelled**)
4. When current incident **State** is **Closed**, then state field should be **Read-only**

Client Script Type

OnLoad

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onLoad() {
    var inc_state = g_form.getValue('state');
    if(inc_state == 1){
        g_form.removeOption('state',2);
        g_form.removeOption('state',3);
        g_form.removeOption('state',6);
        g_form.removeOption('state',7);
        g_form.removeOption('state',8);
    }else
        if(inc_state == 2 || inc_state == 3){
            g_form.removeOption('state',1);
            g_form.removeOption('state',7);
        } else
            if(inc_state == 6){
                g_form.removeOption('state',1);
                g_form.removeOption('state',2);
                g_form.removeOption('state',3);
                g_form.removeOption('state',8);
            } else{
                if(inc_state==7){

g_form.setDisabled('state',true);

                }

            }
    }
}
```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Create New**

3. If current incident state is **New**, then rest of states should not be seen
4. If current incident state is **In-Progress** or **On-Hold**, then **New** and **Closed** States should not visible
5. In **Resolved** state display only **Closed** state value
6. If state is **Closed** State filed should be **Read-only**

Exercise: 8

Requirement:

When incident state is Closed then all fields make read-only

1. If incident state is **closed**, assigned to, assignment group, category, subcategory, impact, urgency, contact type, short description, caller field are make read-only
2. **Attachment Icon** should not see in **Closed** record
3. All **Related Lists** should be hide

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```

function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if(newValue == 7){
        g_form.setDisabled('caller_id',true);

g_form.setDisabled('short_description',true);
        g_form.setDisabled('category',true);
        g_form.setDisabled('subcategory',true);
        g_form.setDisabled('impact',true);
        g_form.setDisabled('urgency',true);
        g_form.setDisabled('assigned_to',true);

g_form.setDisabled('assignment_group',true);
        g_form.setDisabled('contact_type',true);
        g_form.hideRelatedLists(); // Hide
Related Lists
        g_form.disableAttachments(); //Hide
Attachment Icon
    }
}

```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Resolved** Module
3. Change the state value to **Closed** then some field are display in **Read-only**
4. We couldn't see **Attachment Icon** and **Related Lists**

Exercise: 9

Requirement:

When customer changed incident Priority field value then work notes should be mandatory

1. If record is **New** or Form **isLoading** or **oldValue == newValue** same then work notes field should not mandatory
2. When customer changed **Priority 1** to **2** then work notes field make **Mandatory**

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '' ||
g_form.isNewRecord())
        return;
    else if(newValue == oldValue)
        g_form.setMandatory('work_notes',false);
    else
        g_form.setMandatory('work_notes',true);
        g_form.addInfoMessage('If we change
priority value worknotes field mandatory');
}
```

1. Open **New Incident** form check the validation
2. **Incident** → Click on **Create New** Module
3. Change the **Priority** to **1** then **Save** it
4. Again change the **Priority** to **2** then get info message to fill **work notes** mandatory

Exercise: 10

Requirement

User restriction, not allowing update the cell value from Incident List View

1. Any customer tries to change incident state value to **7- Closed** action will be cancel and display alert message to customer
2. Alert Message (**System not allow user to update cell value**)

Client Script Type

OnCellEdit

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onCellEdit(sysIDs, table, oldValues,
newValue, callback) {
    var saveAndClose = true;
    //System cant allow user to update cell
value
    if(newValue == 7){
        alert('System not allowing user to
update cell value');
        saveAndClose = false;
```

```
    } else {  
        saveAndClose = true;  
    }  
    callback(saveAndClose);  
}
```

1. Open **Incident List View** and check the validation
2. **Incident** → Click on **All** Module
3. Change the state value to **7-Closed**
4. Get Error Message (System not allowing user to update cell value)

Exercise: 11

Requirement:

1. When user submit incident form need to display **Assigned to** Name as alert message
2. Give confirmation message also

Client Script Type

OnSubmit

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onSubmit() {  
    var user =  
g_form.getReference('assigned_to');  
    var ticket = confirm('This ticket is  
Assigned to :'+ user.user_name +'. Are you  
sure?');  
    if(ticket == true){  
        return false;  
    }  
}
```

1. Open **New Incident** check the validation
2. Fill **Assigned to** field and **Submit** form
3. Display alert message and and given confirmation message

Exercise: 12

Requirement:

Auto assign the value to Assignment Group depends on category

1. If **Category** is **Network** auto assigned **Network Assignment Group**
2. Category is **Hardware** auto assigned to **Hardware Assignment Group**
3. Category is **Database** auto assigned to **Database Assignment Group**
4. Category is **Software** auto assigned to **Software Assignment Group**

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if (newValue == 'software') {
        g_form.setValue('assignment_group',
'software');
    } else if (newValue == 'hardware') {
        g_form.setValue('assignment_group',
'hardware');
    } else if (newValue == 'network') {
        g_form.setValue('assignment_group',
'network');
    } else if (newValue == 'database') {
        g_form.setValue('assignment_group',
'database');
    }
}
```

1. Open **New Incident** check the validation
2. Select the **Category** is **Software** then ticket is auto assign to **Software Assignment Group**
3. Select **Remaining Category** Also Similarly check the validation

Exercise: 13 Table Incident

Requirement:

Auto set **Caller value** into caller field depends on current logged in user

1. Display alert message say **Hello** message with logged in user full name
2. Caller field should be Read-Only

Client Script Type

OnLoad

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onLoad() {  
    var currentUser = g_user.userID;  
    g_form.setValue('caller_id', currentUser);  
    alert('Hello : ' + g_user.fullName);  
}
```

1. Open **New Incident** check the validation
2. Click on **Create New** module
3. Click on **Ok** button

Exercise: 14

Requirement

Date Validation between two fields for Incident table

1. Create **Two custom fields** in incident tables (**Start Date, End Date**)
2. **Start date** should be always before End date
3. **Start date** should not equal to **End date**
4. Give me **Error Message** under End date field

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition→Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    var startDate =
g_form.getValue('u_start_date');
    var endDate = g_form.getValue('u_end_date');
    if (startDate > endDate || startDate ==
endDate) {
        g_form.showFieldMsg('u_end_date', 'Start
date should not be greater than and equal to end
date', 'error', false);
    }
}
```

}

1. Open **New Incident** check the validation
2. Click on **Create New** module
3. Select a **Start date** that is after the **End date** and check for an error message.
4. Perform another validation: ensure that both dates are not equal.

Exercise: 15

Requirement:

Hide **Resolution Information** section when state is not equal to **Resolved**

1. When Page **Is Loading** and state is not equal to **Resolved** then hide **Resolution Information** section otherwise display section

Client Script Type

OnChange

Procedure

1. Navigate to **System Definition → Client Scripts**
2. Open **Client Scripts**
3. Click on **New**

Script

```
function onChange(control, oldValue, newValue,
isLoading, isTemplate) {
    if (isLoading) {

g_form.setSectionDisplay('resolution_information', false);
```

```
    }  
    if (newValue == 6) {  
  
g_form.setSectionDisplay('resolution_information'  
, true);  
        } else {  
  
g_form.setSectionDisplay('resolution_information'  
, false);  
        }  
    }  
}
```

1. Open **New Incident** check the validation
2. Click on **Create New** module
3. When the form is loaded and the state is not equal to 'Resolved,' hide the resolution information section.